

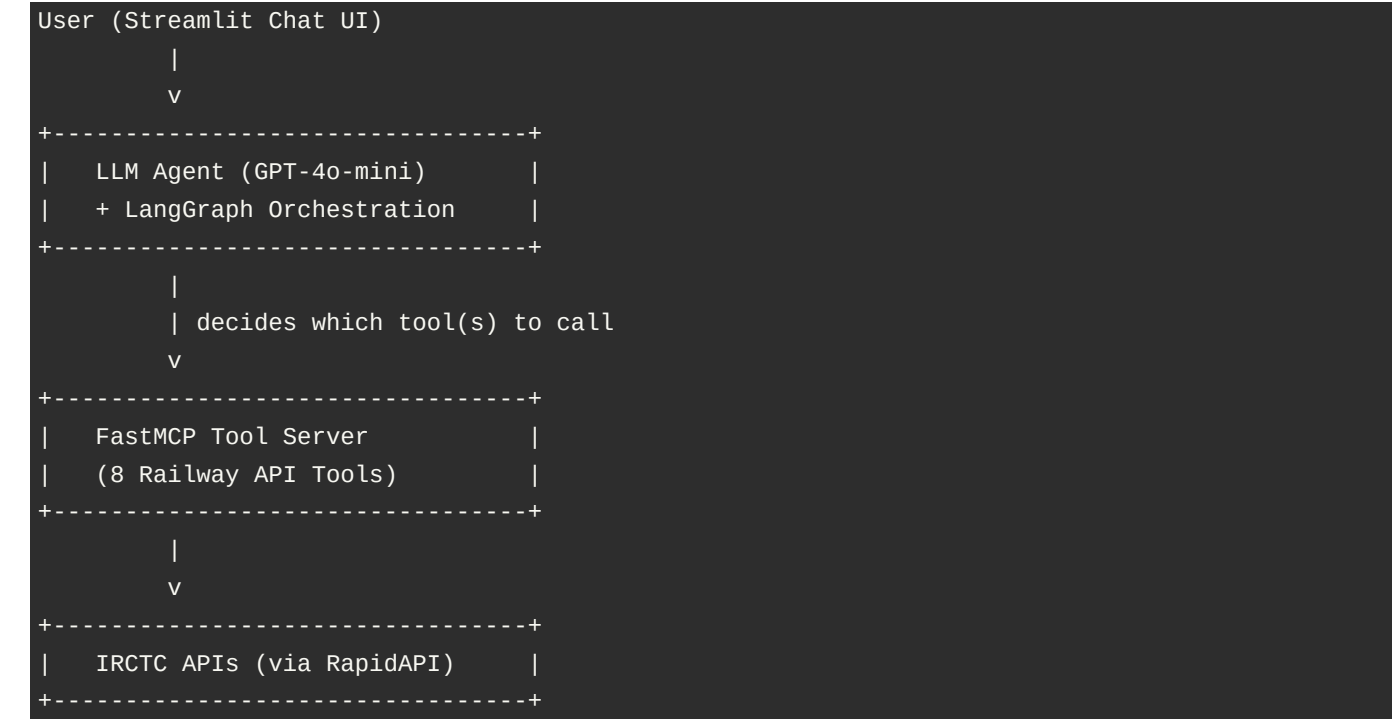
Indian Railways Inquiry AI Assistant

An AI-powered Indian Railways inquiry system that allows users to ask natural language questions and receive accurate, real-time railway information. Built with a tool-driven, agentic AI architecture using LLMs, MCP servers, and LangGraph to ensure reliable, hallucination-free outputs.

Features

Feature	Description
PNR Status	Check booking status with pass
Station Code Lookup	Convert city/station names to
Live Trains	Find trains running between st
Train Schedule	Get complete route/timetable w
Fare Enquiry	Check ticket prices across dif
Live Train Status	Track current location, delay,
Seat Availability	Check class-wise seat availabi
Train Search	Search all trains between two

System Architecture



Project Structure

```
.
+-- RailwayServer.py      # FastMCP tool server with 8 railway API tools
+-- app.py                # Streamlit UI + LangGraph agent orchestration
+-- .env                  # Environment variables (API keys)
+-- requirements.txt      # Python dependencies
```

Tech Stack

Component	Technology
Language	Python 3.10+
Tool Server	FastMCP
Agent Framework	LangChain + LangGraph
LLM	OpenAI GPT-4o-mini
Frontend	Streamlit
Data Source	IRCTC APIs (RapidAPI)

Available Tools

1. ``get_pnr_status``

Fetch detailed PNR status including train info, journey details, and passenger booking/current status.

Input: 10-digit PNR number

2. ``resolve_station_code``

Convert city or station names to official station codes.

Example: "Delhi" -> "NDLS", "Mumbai" -> "BCT"

3. ``get_live_station_trains``

Find trains running between two stations in the next N hours.

Input: Source code, Destination code, Hours (default: 4)

4. ``get_train_schedule``

Get complete route/timetable of a train with all stops, timings, and platform numbers.

Input: 4-5 digit train number

5. ``get_fare``

Get ticket prices for different classes between two stations.

Input: Train number, Source code, Destination code, Date (optional)

6. ``get_live_train_status``

Track current location, delay minutes, and running status of a train.

Input: Train number, Date (optional)

7. ``check_seat_availability``

Check seat availability with class-wise status, fares, and confirmation chances.

Input: Source code, Destination code, Date (DD-MM-YYYY), Train number (optional)

8. ``search_trains``

Search all trains between stations or cities. Supports major city expansion (Delhi searches NDLS, ANVT, DLI, DEE, DEC, SZM).

Input: Source, Destination, Date (optional)

Supported Major Cities

The system automatically searches all major stations for these cities:

City	Stations Searched
Delhi/New Delhi	NDLS, ANVT, DLI, DEE, DEC, SZM
Mumbai	CSMT, BCT, LTT, BDTS
Kolkata	HWH, SDAH, KOAA
Chennai	MAS, MS, MSB

Setup Instructions

1. Clone the Repository

```
git clone https://github.com/KripaAI/railway-inquiry-ai-assistant.git
cd railway-inquiry-ai-assistant
```

2. Create Virtual Environment

```
python -m venv venv

# Windows
venv\Scripts\activate

# macOS/Linux
source venv/bin/activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Configure Environment Variables

Create a .env file in the project root:

```
OPENAI_API_KEY=your_openai_api_key
RAPIDAPI_KEY=your_rapidapi_key
```

Get your API keys:

- * OpenAI API Key: platform.openai.com
- * RapidAPI Key: rapidapi.com/irctc-api

Running the Application

```
streamlit run app.py
```

Open your browser at: <http://localhost:8501>

Example Queries

```
"What is the PNR status of 1234567890?"

"Station code for Varanasi"

"Trains running from NDLS to CNB in the next 4 hours"

"Show me the schedule for train 12565"

"What is the fare from Delhi to Mumbai on train 12951?"

"Check seat availability from HJP to NDLS on 15-01-2026"

"Search trains from Kolkata to Chennai"

"Live status of train 12301"
```

Design Principles

- * Tool-First AI - LLM never guesses railway data; always uses verified API tools
 - * Deterministic Behavior - LangGraph ensures predictable, repeatable agent flows
 - * Separation of Concerns - MCP server handles all API logic independently
 - * Smart Station Handling - Uses station codes directly when provided, resolves only when needed
 - * Production-Ready Architecture - Built with scalability and reliability in mind
-

How It Works

- * User Input - User asks a question in natural language via Streamlit chat
 - * Agent Decision - LangGraph agent analyzes the query and decides which tool(s) to call
 - * Tool Execution - FastMCP server executes the appropriate IRCTC API calls
 - * Response Formatting - Agent formats the API response into a user-friendly answer
 - * Display - Response is shown in the chat interface with proper formatting
-

Future Enhancements

- * Multilingual support (Hindi, regional languages)
 - * Voice input/output integration
 - * Booking recommendations based on availability
 - * Journey planning with connections
 - * Price alerts and notifications
 - * Mobile-responsive PWA version
 - * Caching layer for frequently accessed data
-

Troubleshooting

Issue	Solution
"RAPIDAPI_KEY not found"	Ensure <code>`.env`</code> file exists with
"No trains found"	Verify station codes are corre
Connection timeout	Check internet connection; API
"Train not running today"	Live status only works for tra

License

MIT License

Contributing

Contributions, issues, and feature requests are welcome! Feel free to fork the repository and submit a pull request.

Acknowledgments

- * IRCTC for railway data
- * RapidAPI for API hosting
- * LangChain & LangGraph for agent framework
- * FastMCP for tool server implementation